

Sistema de punto de venta y gestión 360 en la nube

GabAn

Documentación técnica y de tecnología

Arquitectura, stack, modelo de datos, seguridad y despliegue
Versión 1.0

1. Resumen del sistema

GabAn es un sistema de punto de venta y gestión integral (360°) en la nube, con arquitectura **multiempresa** (multi-tenant), **multisucursal** y **multimoneda** (COP, MXN, USD). Integra los módulos de Ventas/Caja, Compras, Productos, Inventarios, Contabilidad, Reportes, Usuarios/Roles, Auditoría y Notificaciones, todos sincronizados entre sí.

El sistema está totalmente contenerizado con Docker y diseñado para ejecutarse en una instancia EC2 de AWS o en cualquier servidor con Docker.

2. Stack tecnológico

Capa	Tecnología	Función
Backend / API	Python 3.12 + FastAPI	API REST asíncrona con documentación OpenAPI automática
ORM / datos	SQLAlchemy 2.0	Mapeo objeto-relacional y consultas
Base de datos	PostgreSQL 16	Almacenamiento; soporta múltiples esquemas (multi-tenant)
Autenticación	JWT (OAuth2) + bcrypt	Tokens sin estado y hashing de contraseñas
Frontend	React 18 + Vite	SPA moderna
Reverse proxy	Nginx	Sirve el frontend y enruta /api al backend
Reportes	openpyxl, reportlab	Exportación a Excel y PDF
Contenedores	Docker + Docker Compose	Empaquetado y orquestación

3. Arquitectura multi-tenant

GabAn usa la estrategia de **un esquema (schema) de PostgreSQL por empresa**. Existe un esquema global **public** con las tablas compartidas, y un esquema **tenant_<slug>** por cada empresa con sus tablas de negocio, físicamente aisladas.

- **Esquema public:** tenants, branches (sucursales), users, roles, currencies, exchange_rates, audit_log.
- **Esquema por tenant:** products, product_variants, inventory, kardex, sales, purchases, suppliers, customers, promotions, contabilidad, bancos, traslados.

En cada petición, un middleware resuelve el tenant a partir del JWT y fija el `search_path` al esquema correspondiente. El `search_path` se reaplica al inicio de cada transacción para garantizar el aislamiento incluso tras confirmaciones (commits).

El aprovisionamiento de una empresa crea automáticamente su esquema, todas sus tablas de negocio (a partir de una plantilla SQL), los roles predefinidos y el usuario administrador.

4. Roles y seguridad

Rol	Alcance	Permisos
MASTER_ADMIN	Todo el sistema	Crear empresas y sucursales, monedas, auditoría global
TENANT_ADMIN	Una empresa	Todos los módulos + usuarios, roles, fiscal y apariencia
VENTAS	Empresa/sucursal	POS, pedidos, promociones, cobros
COMPRAS	Empresa	Proveedores, órdenes, cuentas por pagar
INVENTARIOS	Empresa/sucursal	Ingresos, ajustes, kardex, traslados
PRODUCTOS	Empresa	Alta y edición de catálogo
CONTABILIDAD	Empresa	Asientos, bancos y estados financieros

Permisos por scopes: cada endpoint exige un permiso (por ejemplo `sales:write`, `inventory:read`). Los roles guardan la lista de scopes, de modo que el administrador puede crear roles a la medida sin tocar el código.

- Contraseñas cifradas con `bcrypt`; tokens JWT firmados (HS256) con expiración.
- Aislamiento de datos por esquema y filtro por `search_path`.
- Bitácora de auditoría de todas las acciones (quién, qué, cuándo, IP).
- En producción solo se expone el puerto 80/443; base de datos y backend quedan en la red interna de Docker.
- Idempotencia en ventas (`client_uuid`) para evitar duplicados al sincronizar desde caja offline.

5. Modelo de datos (entidades principales)

Esquema global (public)

Tabla	Descripción
tenants	Empresas; incluye moneda base, país, config fiscal y de apariencia
branches	Sucursales por empresa, con su moneda
users	Usuarios; rol, sucursal, preferencias (modo claro/oscuro)
roles	Roles y su lista de scopes (permisos)
currencies / exchange_rates	Catálogo de monedas y tipos de cambio
audit_log	Bitácora global de acciones

Esquema por tenant

Tabla	Descripción
product_types / categories / brands	Tipos, categorías y marcas de producto
products / product_variants	Referencias y variantes vendibles (talla, color, EAN)
inventory / kardex	Existencias por sucursal e historial inmutable de movimientos
inventory_transfers	Traslados de stock entre sucursales
suppliers / purchase_orders / purchase_order_items	Proveedores y órdenes de compra
supplier_invoices / supplier_payments	Cuentas por pagar y sus pagos
customers / promotions	Clientes y promociones
sales / sale_items / sale_payments	Ventas y pedidos, líneas y pagos (mixtos)
chart_accounts / journal_entries / journal_lines	Plan de cuentas y asientos (partida doble)
bank_accounts / bank_movements	Cuentas bancarias y sus movimientos

6. Integración 360° entre módulos

El corazón de la integración es el inventario y su kardex, actualizados por eventos de todos los módulos dentro de transacciones de base de datos:

- **Compras** → al recibir una orden, aumenta el inventario, escribe el kardex y genera la cuenta por pagar.
- **Ventas/Caja** → al vender, descuenta el inventario y escribe el kardex; el pedido descuenta al entregarse.

- **Inventarios** → ajustes y traslados mueven el stock y quedan registrados en el kardex.
- **Contabilidad y bancos** → cuentas por pagar/cobrar y saldos se enlazan a los documentos que los originan.
- **Reabastecimiento** → compara stock contra mínimos y sugiere/genera órdenes por proveedor preferido.

7. API y módulos del backend

La API expone alrededor de 60 endpoints REST organizados por módulo. La documentación interactiva (OpenAPI/Swagger) está disponible en `/api/docs`.

Prefijo	Módulo
<code>/api/auth</code>	Autenticación (login, JWT)
<code>/api/admin</code>	Master: empresas, sucursales, auditoría
<code>/api/productos</code>	Catálogo, variantes, EAN, categorías
<code>/api/inventarios</code>	Stock, kardex, ajustes, traslados
<code>/api/compras</code>	Proveedores, órdenes, recepción, CxP, reabastecimiento
<code>/api/ventas</code>	POS, pedidos, promociones, clientes, comprobante
<code>/api/contabilidad</code>	Cuentas, asientos, balanza, estados, bancos
<code>/api/reportes</code>	KPIs, gráficos y exportación Excel/PDF
<code>/api/usuarios, /api/roles</code>	Gestión de usuarios y permisos
<code>/api/notificaciones</code>	Alertas transversales
<code>/api/ui-config, /api/preferencias</code>	Apariencia y preferencias

8. Pruebas y calidad

El backend incluye una suite de pruebas automatizadas con `pytest`:

- **Unitarias:** seguridad/JWT, validación contable (partida doble), auditoría, exportadores y contrato de la API.
- **Integración y QA funcional:** con un PostgreSQL real ejercen flujos completos — venta→inventario→kardex, compra→recepción→cuentas por pagar, traslados, idempotencia offline, validaciones y permisos.

```
cd backend
pip install -r requirements-dev.txt
python -m pytest
```

9. Despliegue

El sistema se levanta con Docker Compose. Para producción se usa un override que aísla la base de datos y el backend, y reinicia los servicios automáticamente.

```
# Local (pruebas)
docker compose up --build

# Producción (EC2)
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

Para el detalle paso a paso en AWS EC2, consulte el documento **Guía de despliegue en AWS EC2**.